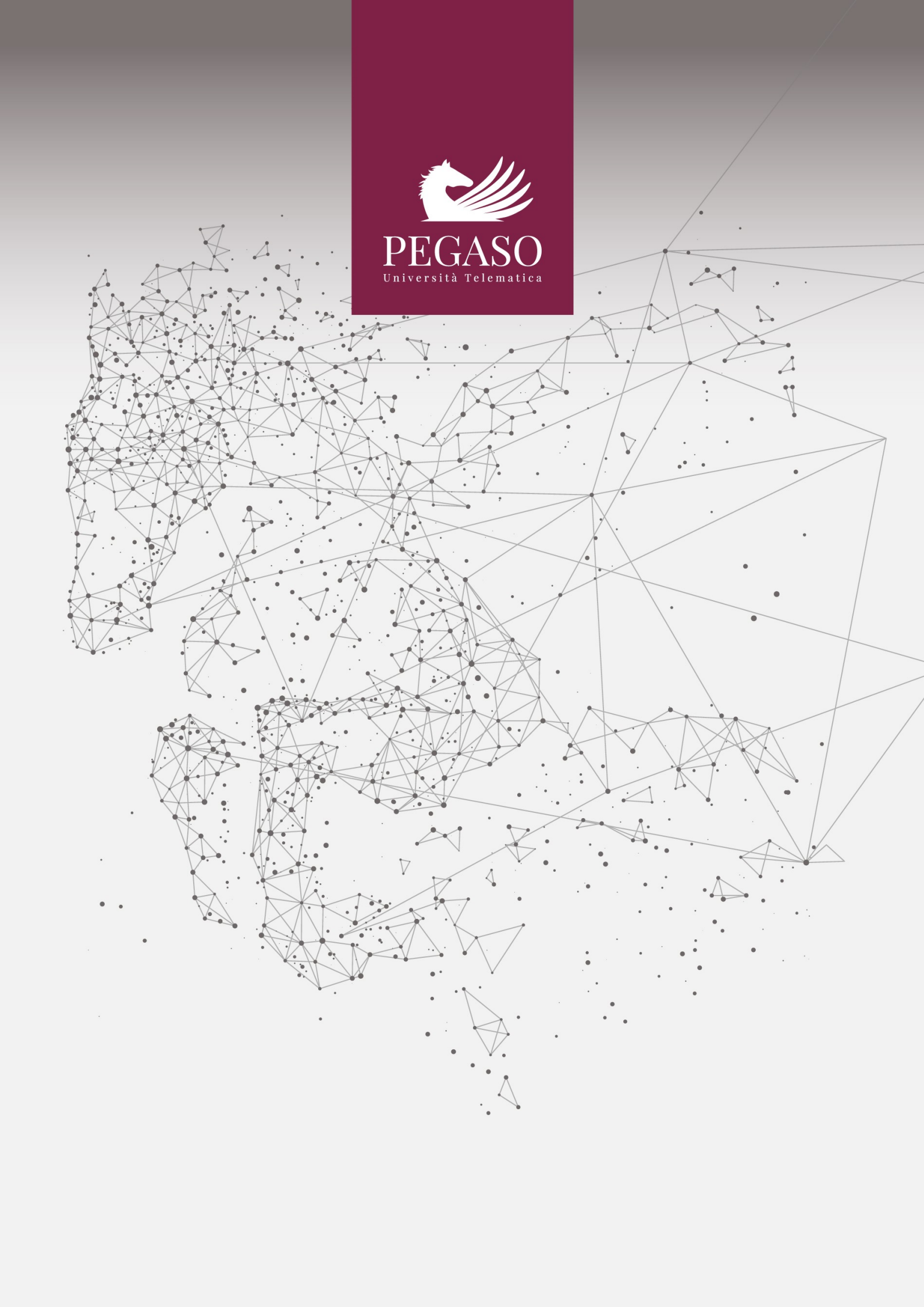

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. JAVASCRIPT BASICS....3
2. FUNZIONE....6
3. JAVASCRIPT AND JSON....9
4. JS HTML DOM....11
BIBLIOGRAFIA SITOGRAFIA....15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Javascript Basics

Abbiamo 3 possibilità di inserire tale script nella pagina:

1. Nell'head (posizione 1)
2. Nel body subito dopo il tag body (posizione 2)
3. In un punto qualunque del body (posizione 3)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>TEST</title>
  <!-- POSIZIONE 1 -->
</head>
<body>
  <!-- POSIZIONE 2 -->
  <p id="myP">Hello world</p>
  <!-- POSIZIONE 3 -->
</body>
</html>
```

1. Se lo script si trova nell'head, questo viene invocato immediatamente; se questo dipende da elementi specifici della pagina HTML lo script potrebbe non funzionare correttamente
2. Se lo script si trova nel body, questo viene invocato immediatamente durante il caricamento della pagina rispettando il suo posizionamento; se lo script dipende da elementi specifici della pagina HTML che si trovano DOPO (a livello di posizionamento) il caricamento dello script, lo script potrebbe non funzionare correttamente
3. Se lo script si trova nel body subito prima la chiusura del tag stesso, siamo certi che tutti gli elementi della pagina sono stati caricati, pertanto se lo script dipende da elementi specifici della pagina HTM, siamo certi che lo script funzionerà correttamente

È inoltre possibile scrivere il codice Javascript all'interno di un file esterno (tipicamente con estensione .js) e richiamarlo in una delle posizioni indicate in precedenza, attraverso la sintassi:

```
<script src="script.js"></script>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La sintassi JavaScript definisce due tipi di valori:

- Fissi (fixed)
- Variabili (variables)

I valori fissi sono chiamati letterali (literals) mentre quelli variabili sono chiamati variabili (variabiles). Le due regole di sintassi più importanti per i valori fissi sono:

- I numeri sono scritti con o senza decimali
- Le stringhe sono scritte tra singoli i doppi apici

In JavaScript, sia la parola chiave **var** che la parola chiave **let** vengono utilizzate per dichiarare variabili, ma hanno alcune differenze importanti. La principale differenza tra var e let è che le variabili dichiarate con var hanno un ambito di funzione (function scope), mentre le variabili dichiarate con let hanno un ambito di blocco (block scope). Altre peculiarità del let sono:

- le variabili definite con let non possono essere ridefinite
- le variabili definite con let devono essere dichiarate prima dell'utilizzo

JavaScript ha 8 tipi di dato: String, Number, Bigint, Boolean, Undefined, Null, Symbol, Object

Le istruzioni condizionali vengono utilizzate per eseguire azioni diverse in base a diverse condizioni.

In JavaScript abbiamo le seguenti istruzioni condizionali:

- Usare **if** per specificare un blocco di codice da eseguire, se una condizione specificata è vera
- Usare **else** per specificare un blocco di codice da eseguire, se la stessa condizione è falsa
- Usare **else if** per specificare una nuova condizione da testare, se la prima condizione è falsa
- Usare **switch** per specificare molti blocchi alternativi di codice da eseguire

JavaScript supporta diversi tipi di cicli:

- **for** - cicla attraverso un blocco di codice un numero di volte
- **for/in** - cicla attraverso le proprietà di un oggetto
- **for/of** - cicla attraverso i valori di un oggetto iterabile

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **while** - cicla attraverso un blocco di codice finché una specifica condizione è vera
- **do/while** - cicla anche attraverso un blocco di codice finché una specifica condizione è vera

In JavaScript, un **oggetto** è una struttura di dati complessa e flessibile utilizzata per rappresentare e organizzare i dati. È un contenitore per un insieme di proprietà (che possono essere variabili, funzioni, altri oggetti, ecc.) che possono essere accedute e manipolate tramite il nome della proprietà o l'indice.

Gli oggetti sono spesso utilizzati per rappresentare entità del mondo reale come persone, automobili, case, ecc., ma possono essere utilizzati anche per organizzare il codice e definire moduli e librerie riutilizzabili. Gli oggetti in JavaScript sono dinamici, il che significa che le proprietà possono essere aggiunte o rimosse in qualsiasi momento durante l'esecuzione del programma.

Per accedere alle proprietà di un oggetto in JavaScript, si può utilizzare la notazione punto (**dot notation**) o la notazione delle parentesi quadre (**bracket notation**).

Un **array** è un oggetto che permette di memorizzare una sequenza di valori in una singola variabile. Un array può contenere valori di qualsiasi tipo di dato, come numeri, stringhe, oggetti o altri array.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Funzione

In JavaScript, le funzioni sono un costrutto fondamentale del linguaggio e permettono di scrivere codice modulare, riutilizzabile e organizzato. Una funzione in JavaScript è un blocco di codice che viene eseguito quando viene chiamato (o invocato) e può ricevere uno o più parametri.

La sintassi per definire una funzione è la seguente:

```
function nomeFunzione(parametro1, parametro2, ...) {
  // blocco di codice da eseguire
}
```

Il nome della funzione può essere scelto dal programmatore e deve seguire le stesse regole dei nomi di variabili: può contenere lettere, numeri e il carattere underscore, ma non può iniziare con un numero. I parametri sono le variabili che la funzione accetta in input e possono essere utilizzati all'interno del blocco di codice.

Per chiamare una funzione, si utilizza il nome della funzione seguito da parentesi tonde e eventuali argomenti separati da virgole. Ad esempio:

```
nomeFunzione(argomento1, argomento2, ...);
```

Una funzione può restituire un valore utilizzando la keyword return. Il valore restituito può essere di qualsiasi tipo (numero, stringa, booleano, oggetto, array, etc.) e può essere utilizzato per assegnare il valore di una variabile o come input per altre funzioni.

Le funzioni possono anche essere assegnate a una variabile o essere definite come funzioni anonime (senza un nome). Ad esempio:

```
// definizione di una funzione anonima
const quadrato = function(x) {
  return x * x;
};

// definizione di una funzione assegnata a una variabile
const somma = function(a, b) {
  return a + b;
};
```

Le funzioni in JavaScript possono essere usate in molti modi, come ad esempio:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- per definire comportamenti di eventi (come la pressione di un bottone o la scelta di un'opzione in un menu)
- per organizzare il codice in moduli e facilitare la manutenzione
- per definire metodi su oggetti e classi
- per creare e manipolare array e oggetti
- per effettuare chiamate HTTP tramite AJAX
- […]

Le **arrow function** sono un'alternativa sintattica alle funzioni tradizionali in JavaScript. Sono state introdotte a partire da ECMAScript 6 e permettono di definire funzioni in modo più conciso, leggibile e intuitivo.

Ecco la sintassi di base delle arrow function:

```
(parametro1, parametro2, ...) => {
  // blocco di codice da eseguire
}
```

Invece di utilizzare la keyword function per definire una funzione, si utilizza la sintassi

```
() => {}
```

La funzione può avere uno o più parametri tra parentesi tonde, separati da virgola. Il blocco di codice viene racchiuso tra parentesi graffe, proprio come in una funzione tradizionale.

Le arrow function hanno alcune caratteristiche peculiari che le rendono differenti dalle funzioni tradizionali. In particolare:

- Non hanno un proprio this: quando una variabile viene usata all'interno di una arrow function, il valore di this è preso dal contesto in cui viene definita la funzione.
- Non hanno un oggetto arguments: le arrow function non dispongono di un oggetto arguments, ma possono accedere ai parametri tramite la sintassi (...args) => {}.
- Sono più concise: grazie alla loro sintassi concisa, le arrow function sono spesso più leggibili e intuitive delle funzioni tradizionali.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco un esempio di utilizzo delle arrow function:

```
// funzione tradizionale
function quadrato(x) {
  return x * x;
}

// arrow function equivalente
const quadrato = (x) => x * x;
```

In questo esempio, viene definita una funzione tradizionale quadrato che riceve un parametro x e ritorna il quadrato di x. La stessa funzione viene poi definita utilizzando una arrow function più concisa, in cui viene specificato il parametro x seguito dalla sintassi => e dal blocco di codice che calcola il quadrato di x.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Javascript and Json

In JavaScript, è possibile convertire oggetti e array in formato JSON utilizzando i metodi JSON.stringify() e JSON.parse(). Questi metodi sono disponibili nativamente nel browser e possono essere utilizzati per la conversione da e verso JSON.

La conversione da oggetto o array a JSON viene effettuata utilizzando il metodo JSON.stringify(), che accetta come parametro l'oggetto o l'array da convertire e restituisce una stringa JSON. Ad esempio:

```
const persona = {
  nome: "Mario",
  cognome: "Rossi",
  eta: 30
};

const personaJson = JSON.stringify(persona);

console.log(personaJson);
// Output: "{"nome":"Mario","cognome":"Rossi","eta":30}"
```

In questo esempio, l'oggetto persona viene convertito in una stringa JSON utilizzando il metodo JSON.stringify() e il risultato viene assegnato alla variabile personaJson.

La conversione da JSON a oggetto o array viene effettuata utilizzando il metodo JSON.parse(), che accetta come parametro la stringa JSON da convertire e restituisce l'oggetto o l'array corrispondente. Ad esempio:

```
const personaJson = '{"nome":"Mario","cognome":"Rossi","eta":30}';
const persona = JSON.parse(personaJson);
console.log(persona);
// Output: {nome: "Mario", cognome: "Rossi", eta: 30}
```

In questo esempio, la stringa JSON personaJson viene convertita in un oggetto utilizzando il metodo JSON.parse() e il risultato viene assegnato alla variabile persona.

È importante notare che i metodi JSON.stringify() e JSON.parse() supportano solo una selezione limitata di tipi di dati JavaScript. In particolare, gli oggetti che includono funzioni, proprietà non numeriche o cicli di riferimento non possono essere convertiti in formato JSON.

Supponiamo di avere il seguente json:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
{
  "nome": "Mario",
  "cognome": "Rossi",
  "eta": 30,
  "hobbies": [
    "giocare a calcio",
    "leggere libri"
  ],
  "indirizzo": {
    "via": "Via Roma",
    "citta": "Milano"
  }
}
```

Per accedere ai dati di questo file JSON tramite JavaScript, possiamo utilizzare la funzione JSON.parse() per convertire il file JSON in un oggetto JavaScript:

```
var jsonFile = '{"nome": "Mario", […]

var data = JSON.parse(jsonFile);

console.log(data.nome); // "Mario"
console.log(data.hobbies[0]); // "giocare a calcio"
console.log(data.indirizzo.citta); // "Milano"
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. JS HTML DOM

Il JavaScript HTML DOM (Document Object Model) è un'interfaccia di programmazione delle applicazioni (API) che consente di manipolare gli elementi HTML e XML di una pagina web attraverso il linguaggio di programmazione JavaScript. In pratica, il DOM rappresenta l'albero di struttura della pagina web e ogni elemento è rappresentato da un oggetto del DOM.

Grazie al JavaScript HTML DOM, è possibile accedere, creare e modificare gli elementi di una pagina web dinamicamente, senza dover ricaricare la pagina stessa: questo consente di creare interazioni dinamiche con gli utenti, animazioni, applicazioni web interattive e molto altro ancora.

Per accedere agli elementi del DOM, si utilizzano i metodi come:

- getElementById
- getElementsByClassName
- getElementsByTagName
- querySelector
- querySelectorAll

Una volta acceduto all'elemento desiderato, è possibile manipolarlo modificando le sue proprietà, come ad esempio innerHTML, textContent, style e classList.

Inoltre, è possibile aggiungere eventi a un elemento utilizzando il metodo addEventListener e creare nuovi elementi HTML utilizzando il metodo createElement. Grazie a queste funzionalità, è possibile creare pagine web dinamiche e interattive che rispondono alle azioni degli utenti.

Analizziamo i metodi per accedere agli elementi del DOM:

- **getElementById()**: questo metodo restituisce l'elemento HTML che ha un attributo "id" corrispondente al valore specificato come argomento.

```
var elemento = document.getElementById("idElemento");
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **getElementsByClassName()**: questo metodo restituisce una collezione di tutti gli elementi HTML che hanno una classe CSS corrispondente al valore specificato come argomento.

```
var elementi=document.getElementsByClassName("classElem");
```

- **getElementsByTagName()**: questo metodo restituisce una collezione di tutti gli elementi HTML che hanno un tag HTML corrispondente al valore specificato come argomento.

```
var elementi = document.getElementsByTagName("tagElem");
```

- **querySelector()**: questo metodo restituisce il primo elemento HTML che corrisponde al selettore CSS specificato come argomento.

```
var elemento=document.querySelector("#idElem .classElem");
```

- **querySelectorAll()**: questo metodo restituisce una collezione di tutti gli elementi HTML che corrispondono al selettore CSS specificato come argomento.

```
var elementi=document.querySelectorAll(".classeElem");
```

Riportiamo di seguito quali sono i selettori CSS che possono essere utilizzati per poter accedere al DOM della pagina.

Selettore di tipo

Il selettore di tipo seleziona tutti gli elementi di un determinato tipo HTML; ad esempio, il selettore di tipo "p" selezionerà tutti gli elementi di paragrafo nella pagina. La sintassi è il nome del tipo di elemento, ad esempio "p" o "h1".

Selettore di classe

Il selettore di classe seleziona tutti gli elementi che hanno una determinata classe HTML; le classi vengono definite utilizzando l'attributo "class" negli elementi HTML e in CSS vengono indicate con un punto ".". Ad esempio, il selettore ".intro" selezionerà tutti gli elementi con la classe "intro". La sintassi è il punto "." seguito dal nome della classe, ad esempio ".intro".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Selettore di ID

Il selettore di ID seleziona un singolo elemento con un determinato ID HTML; gli ID vengono definiti utilizzando l'attributo "id" negli elementi HTML e in CSS vengono indicati con un cancelletto "#". Ad esempio, il selettore "#header" selezionerà l'elemento con l'ID "header". La sintassi è il cancelletto "#" seguito dal nome dell'ID, ad esempio "#header".

Selettore discendente

Il selettore discendente seleziona tutti gli elementi figli di un altro elemento; la sintassi è quella di elencare gli elementi separati da uno spazio. Ad esempio, il selettore "ul li" selezionerà tutti gli elementi "li" che sono figli di un elemento "ul".

Selettore di figlio diretto

Il selettore di figlio diretto seleziona solo gli elementi che sono figli diretti di un altro elemento; la sintassi è quella di elencare gli elementi separati da ">", ad esempio "ul > li" selezionerà solo gli elementi "li" che sono figli diretti di un elemento "ul".

Selettore di attributo

Il selettore di attributo seleziona gli elementi in base a un attributo HTML specifico. La sintassi è quella di scrivere il nome dell'attributo tra parentesi quadre, ad esempio "[href]" selezionerà tutti gli elementi che hanno un attributo "href".

Selettore di attributo con valore

Il selettore di attributo con valore seleziona gli elementi in base a un attributo HTML specifico e al suo valore. La sintassi è quella di scrivere il nome dell'attributo seguito dall'operatore di uguaglianza "=" e dal valore desiderato tra virgolette, ad esempio "[href='#']" selezionerà tutti gli elementi che hanno un attributo "href" con valore "#".

Selettore di pseudo-classe

Il selettore di pseudo-classe seleziona gli elementi in base a uno stato o una condizione specifici, come ad esempio quando l'utente posiziona il mouse sopra l'elemento o quando è stato visitato un link. La

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sintassi è quella di scrivere il nome della pseudo-classe preceduto da due punti, ad esempio ":hover" selezionerà l'elemento quando l'utente posiziona il mouse sopra di esso.

Selettore di pseudo-elemento

Il selettore di pseudo-elemento seleziona parti specifiche di un elemento HTML, come ad esempio la prima lettera di un paragrafo o la prima riga di un blocco di testo. La sintassi è quella di scrivere il nome del pseudo-elemento preceduto da due punti, ad esempio "::first-letter" selezionerà la prima lettera di ogni elemento selezionato.

Riportiamo alcuni esempi di gestione delle proprietà di elementi del DOM usando Javascript:

```
// seleziona l'elemento con l'ID "header"
var header = document.getElementById("header");

// modifica il CSS "background-color" dell'elemento selezionato
header.style.backgroundColor = "blue";

// seleziona tutti gli elementi con la classe "box"
var boxes = document.getElementsByClassName("box");

// modifica il CSS "border" di tutti gli elementi selezionati
for (var i = 0; i < boxes.length; i++) {
  boxes[i].style.border = "1px solid black";
}

// seleziona gli elementi p figli diretti di un elemento con l'ID "main"
var paragraphs = document.querySelectorAll("#main > p");

// modifica il CSS "font-size" di tutti gli elementi selezionati
for (var i = 0; i < paragraphs.length; i++) {
  paragraphs[i].style.fontSize = "16px";
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. *Introduzione alle Tecnologie Web*, McGraw-Hill, 2010
- A. S. Tanembaum. *Computer Networks* (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. *Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR*, 2005
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).